

Datos del estudiante		Información
Asignatura		Fundamentos de Programación
Tema		Vectores y matrices en lenguaje C
Nombre del estudiante		Dicson Alexander Tigse Tigse
NRC/Paralelo		28561
Fecha		20/05/26

1. Orientaciones generales

En estos ejercicios se trabaja con estructuras repetitivas, arreglos unidimensionales y arreglos bidimensionales. Antes de pasar a Code::Blocks, conviene revisar cuatro elementos: el enunciado, la tabla de objetos, la lógica de recorrido y la prueba de escritorio.

Etapas	Qué debe hacer el estudiante	Pregunta guía
1. Comprender el enunciado	Leer el problema e identificar qué se pide calcular o mostrar.	¿Cuál es la salida esperada?
2. Identificar objetos	Reconocer vectores, matrices, contadores, acumuladores, banderas y variables auxiliares.	¿Qué datos entran, qué datos se procesan y qué datos salen?
3. Diseñar la lógica	Definir recorridos con ciclos repetitivos, condiciones y asignaciones.	¿Recorro de inicio a fin, de fin a inicio, por filas o por columnas?
4. Probar la solución	Hacer una prueba de escritorio con pocos datos.	¿La solución funciona en un caso pequeño y claro?
5. Codificar y validar	Pasar la lógica a C en Code::Blocks y verificar la salida.	¿El código compila y el resultado coincide con la prueba?

Tip amigable

Cuando veas un vector piensa en una fila de casilleros. Cuando veas una matriz piensa en una tabla con filas y columnas. Los contadores *i* y *j* ayudan a recorrer esos casilleros de forma ordenada.

2. Ejercicios de vectores

Ejercicio 2.1.1. Vector al revés

Enunciado. Desarrolle un programa que lea las *N* componentes de un vector y las escriba a continuación en orden inverso al introducido. El valor de *N* se introduce por teclado y se supone que será igual o menor a 10.

Idea clave. Leer el vector en orden normal y luego recorrerlo desde la última posición hasta la primera.

Objeto	Nombre	Valor	Tipo
Vector de 10 componentes	vec	var	real
Contador	i	var	entero
Uno	1	cte	entero
Número de componentes del vector	N	var	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué el segundo recorrido debe decrementar el índice? ¿Qué pasaría si se imprime nuevamente de 1 a <i>N</i> ?	

Tabla de objetos:

Objeto	Nombre	Tipo
Vector de 10 componentes	vec	entero
Contador	i	entero
Número de componentes	N	entero

Código en C

```
#include <stdio.h>

int main() {
    int vec[10];
    int i, N;

    printf("Ingrese el numero de componentes: ");
    scanf("%d", &N);

    // Lectura del vector
    for(i = 0; i < N; i++) {
        printf("Ingrese el elemento %d: ", i + 1);
        scanf("%d", &vec[i]);
    }

    // Mostrar vector al revés
    printf("\nVector en orden inverso:\n");
    for(i = N - 1; i >= 0; i--) {
        printf("%d ", vec[i]);
    }

    return 0;
}
```

Pseudocódigo

Inicio

Leer N

Para i = 0 hasta N-1 hacer

 Leer vec[i]

Fin Para

Para i = N-1 hasta 0 hacer

 Escribir vec[i]

Fin Para

Fin

Prueba de escritorio

Entrada:

$N = 5$

$\text{vec} = [1, 3, 5, 7, 9]$

Proceso:

$i=4 \rightarrow 9$

$i=3 \rightarrow 7$

$i=2 \rightarrow 5$

$i=1 \rightarrow 3$

$i=0 \rightarrow 1$

Salida:

9 7 5 3 1

Reflexión

En este ejercicio se usa un recorrido normal para guardar los datos y otro al revés para mostrarlos invertidos. Si se recorriera igual en ambos casos, los números saldrían en el mismo orden y no cambiaría nada.

Ejercicio 2.1.2.

Norma de un vector al cuadrado

Enunciado. Desarrolle un programa que lea las n componentes reales de un vector vec y calcule la norma al cuadrado: $m = \text{vec}^T \times \text{vec} = \text{suma de } \text{vec}(i)^2$. El tamaño n se introduce por teclado y puede suponerse igual o menor que 10.

Idea clave. Inicializar res en 0 y acumular el cuadrado de cada componente leída.

Objeto	Nombre	Valor	Tipo
Vector de 10 componentes	vec	var	real
Contador	i	var	entero
Número de componentes del vector	n	var	entero
Norma al cuadrado	res	var	real
Uno	1	cte	entero
Objeto		Valor	Tipo
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué crees debe iniciar en cero? ¿Qué error aparecería si se usa un valor basura no inicializado?	

Tabla de objetos:

Objeto	Nombre	Tipo
Vector de 10 componentes	vec	real
Contador	i	entero
Número de componentes	N	entero
Norma al cuadrado	res	real

Pseudocódigo

Inicio

Leer N

res = 0

Para i = 0 hasta N-1 hacer

 Leer vec[i]

 res = res + vec[i]^2

Fin Para

Escribir res

Fin

Código C

```
#include <stdio.h>
```

```
int main() {
```

```
    float vec[10], res = 0;
```

```
    int N, i;
```

```
    printf("Ingrese el numero de componentes: ");
```

```
    scanf("%d",&N);
```

```
    for(i=0; i<N; i++) {
```

```
        printf("Ingrese un valor: ");
```

```
        scanf("%f",&vec[i]);
```

```

        res = res + (vec[i]*vec[i]);
    }
    printf("La norma al cuadrado es: %.2f", res);
    return 0;
}

```

Prueba de escritorio

Entrada:

$N = 3$

$\text{vec} = [2, 4, 1]$

Proceso:

$i=0 \rightarrow \text{res}=0+(2^2)=4$

$i=1 \rightarrow \text{res}=4+(4^2)=20$

$i=2 \rightarrow \text{res}=20+(1^2)=21$

Salida:

21

Reflexión:

La variable res inicia en 0 porque va guardando la suma de los cuadrados. Así el cálculo sale correcto desde el principio.

Ejercicio 2.1.3.

Vector con término general dado

Enunciado. Sea la sucesión $v_k = k^2 + 3$. Desarrolle un programa que lea el número n de componentes que se quieren de la sucesión y las almacene en un vector vec , de forma que $\text{vec}(i) = v_i$. Puede asumir que n será menor o igual a 100.

Idea clave. Usar un ciclo para calcular cada término de la sucesión y almacenarlo en la posición correspondiente del vector.

Objeto	Nombre	Valor	Tipo
Vector de 100 componentes	vec	var	real
Contador	i	var	entero
Uno	1	cte	entero
Número de componentes del vector	n	var	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué diferencia existe entre el índice usado en el enunciado y el índice que usa C cuando empieza en 0?	

Tabla de objetos:

Objeto	Nombre	Tipo
Vector de 100 componentes	vec	real
Contador	i	entero
Número de componentes	N	entero

Pseudocódigo

Inicio
Leer N
Para i = 0 hasta N-1 hacer
 $\text{vec}[i] = (i+1)^2 + 3$
Fin Para
Escribir vec
Fin

Código en C

```
#include <stdio.h>
int main() {
    float vec[100];
    int N, i;
    printf("Ingrese el numero de componentes: ");
    scanf("%d",&N);
    for(i=0; i<N; i++) {
        vec[i] = (i+1)*(i+1)+3;
    }
    printf("Vector generado:\n");
    for(i=0; i<N; i++) {
        printf("%.0f ", vec[i]);
    }
    return 0;
}
```

Prueba de escritorio

Entrada:

N = 5

Proceso:

i=0 → $\text{vec}[0] = (1^2) + 3 = 4$

i=1 → $\text{vec}[1] = (2^2) + 3 = 7$

i=2 → $\text{vec}[2] = (3^2) + 3 = 12$

i=3 → $\text{vec}[3] = (4^2) + 3 = 19$

i=4 → $\text{vec}[4] = (5^2) + 3 = 28$

Salida:

4 7 12 19 28

Reflexión crítica:

En este ejercicio se usa un ciclo para calcular cada valor de la sucesión y guardarlo en el vector. Se usa $i+1$ porque en C los índices empiezan en 0 y la fórmula del ejercicio empieza en 1.

Ejercicio 2.1.4.

Comprobar si dos valores pertenecen a un vector

Enunciado. Realice un algoritmo que lea dos números enteros por teclado y determine si ambos valores forman parte de un vector de enteros previamente definido de dimensión 15.

Idea clave. Usar dos variables bandera para marcar si num1 y num2 se encuentran en el vector.

Objeto	Nombre	Valor	Tipo
Número de componentes del vector	N	cte	entero
Vector de 15 números	vec	var	entero
Variable auxiliar para el primer dato	aux1	var	entero
Variable auxiliar para el segundo dato	aux2	var	entero
Contador	i	var	entero
Primer dato de entrada	num1	var	entero
Segundo dato de entrada	num2	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué se necesitan dos banderas y no una sola? ¿Qué condición final permite afirmar que ambos números están en el vector?	

Tabla de objetos:

Objeto	Nombre	Tipo
Vector de 15 números	vec	entero
Primer dato	num1	entero
Segundo dato	num2	entero
Variable auxiliar 1	aux1	entero
Variable auxiliar 2	aux2	entero
Contador	i	entero

Pseudocódigo

```
Inicio
aux1 = 0
aux2 = 0
Leer num1
Leer num2
Para i = 0 hasta 14 hacer
    Si vec[i] == num1 entonces
        aux1 = 1
    Fin Si
    Si vec[i] == num2 entonces
        aux2 = 1
    Fin Si
Fin Para
Si aux1 == 1 y aux2 == 1 entonces
    Escribir "Ambos numeros pertenecen al vector"
Sino
    Escribir "No pertenecen ambos numeros al vector"
Fin Si
Fin
```

Código en C

```
#include <stdio.h>
int main() {
    int vec[15]={2,5,8,10,12,15,18,20,22,25,30,35,40,45,50};
    int num1, num2, aux1=0, aux2=0, i;
    printf("Ingrese el primer numero: ");
    scanf("%d",&num1);
    printf("Ingrese el segundo numero: ");
    scanf("%d",&num2);
    for(i=0; i<15; i++) {
        if(vec[i]==num1) {
            aux1=1;
        }
        if(vec[i]==num2) {
            aux2=1;
        }
    }

    if(aux1==1 && aux2==1) {
        printf("Ambos numeros pertenecen al vector");
    } else {
```



```

        printf("No pertenecen ambos numeros al vector");
    }
    return 0;
}

```

Prueba de escritorio

Entrada:

num1 = 10

num2 = 25

Proceso:

aux1=0

aux2=0

i=3 → vec[3]=10 → aux1=1

i=9 → vec[9]=25 → aux2=1

Salida:

Ambos numeros pertenecen al vector

Reflexión:

En este ejercicio se usan dos variables auxiliares para comprobar cada número por separado. Así se puede saber si los dos valores están dentro del vector o no.

Ejercicio 2.1.5.

Vector de factoriales

Enunciado. Dado un vector Vec que contiene los primeros 15 números naturales, calcule un vector fact con sus factoriales y muéstrelo por pantalla. Considere el problema de almacenamiento a partir de factoriales grandes.

Idea clave. Usar ciclos anidados: uno para recorrer los números y otro para calcular el factorial de cada componente.

Objeto	Nombre	Valor	Tipo
Vector de 15 números	Vec	var	entero
Vector de 15 números	fact	var	entero
Objeto	Nombre	Valor	Tipo
Contador	i	var	entero
Contador	j	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Quince	15	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué el factorial crece tan rápido? ¿Qué tipo de dato conviene usar si el resultado supera la capacidad de int?	

Tabla de objetos:

Objeto	Nombre	Tipo
Vector de 15 números	Vec	entero
Vector de factoriales	fact	entero
Contador	i	entero
Contador	j	entero

Pseudocódigo

Inicio

Para i = 0 hasta 14 hacer

 Vec[i] = i + 1

Fin Para

Para i = 0 hasta 14 hacer

 fact[i] = 1

 Para j = 1 hasta Vec[i] hacer

 fact[i] = fact[i] * j

 Fin Para

Fin Para

Escribir fact

Fin

Código en C

```
#include <stdio.h>
```

```
int main() {
```

```
    int Vec[15], fact[15];
```

```
    int i, j;
```

```
    for(i=0; i<15; i++) {
```

```
        Vec[i] = i + 1;
```

```
    }
```

```
    for(i=0; i<15; i++) {
```

```
        fact[i] = 1;
```

```
        for(j=1; j<=Vec[i]; j++) {
```

```
            fact[i] = fact[i] * j;
```

```
        }
```

```
    }
```

```
    printf("Vector de factoriales:\n");
```

```
    for(i=0; i<15; i++) {
```

```
        printf("%d ", fact[i]);
```

```
    }
```

```
    return 0;
```

```
}
```

Prueba de escritorio

Proceso:

Vec = [1,2,3,4,5]

1! = 1

2! = 2

3! = 6

4! = 24

5! = 120

Salida:

1 2 6 24 120

Reflexión:

En este ejercicio se usan dos ciclos: uno para recorrer los números y otro para calcular cada factorial. El factorial crece rápido, por eso con números grandes hay que tener cuidado con el tipo de dato.

3. Ejercicios de matrices**Ejercicio 2.2.1.****Suma de componentes de una matriz**

Enunciado. Halle la suma de las componentes de una matriz cuya dimensión y componentes se leen por teclado y por filas. Suponga que el número de filas N y columnas M son menores o iguales a 10.

Idea clave. Recorrer la matriz por filas con dos ciclos anidados y acumular cada componente en res.

Objeto	Nombre	Valor	Tipo
Matriz de 10x10 componentes	mat	var	real
Contador	i	var	entero
Contador	j	var	entero
Número de filas de la matriz	N	var	entero
Número de columnas de la matriz	M	var	entero
Suma de las componentes	res	var	real
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué se necesita un ciclo para filas y otro para columnas? ¿Qué representa cada índice?	

Tabla de objetos:

Objeto	Nombre	Tipo
Matriz de 10×10 componentes	mat	real
Contador	i	entero
Contador	j	entero
Número de filas	N	entero
Número de columnas	M	entero
Suma de componentes	res	real

Pseudocódigo

```
Inicio
Leer N
Leer M
res = 0
Para i = 0 hasta N-1 hacer
    Para j = 0 hasta M-1 hacer
        Leer mat[i][j]
        res = res + mat[i][j]
    Fin Para
Fin Para
Escribir res
Fin
```

Código en C

```
#include <stdio.h>
int main() {
    float mat[10][10], res = 0;
    int N, M, i, j;
    printf("Ingrese numero de filas: ");
    scanf("%d",&N);
    printf("Ingrese numero de columnas: ");
    scanf("%d",&M);
    for(i=0; i<N; i++) {
        for(j=0; j<M; j++) {
            printf("Ingrese un valor: ");
            scanf("%f",&mat[i][j]);
            res = res + mat[i][j];
        }
    }
    printf("La suma de la matriz es: %.2f", res);

    return 0;
}
```

Prueba de escritorio

Entrada:

N = 2

M = 2

1 2

3 4

Proceso:

res=0

res=0+1=1

res=1+2=3

res=3+3=6

res=6+4=10

Salida:

10

Reflexión:

En este ejercicio se usan dos ciclos porque la matriz tiene filas y columnas. Un ciclo recorre las filas y el otro las columnas para poder sumar todos los elementos.

Ejercicio 2.2.2.

Escritura de matriz en sentido inverso

Enunciado. Dada una matriz de N x N elementos, realice un algoritmo que recorra la matriz por filas desde la última a la primera y cada fila en sentido inverso, desde la última columna a la primera, mostrando cada elemento.

Idea clave. Inicializar los índices en la última posición y decrementar hasta llegar a la primera.

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz 4 x 4	mat	var	entero
Contador	i	var	entero
Contador	j	var	entero
Cero	0	cte	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué cambia entre recorrer una matriz normalmente y recorrerla al revés? ¿Qué riesgo aparece con los límites de los índices?	

Tabla de objetos:

Objeto	Nombre	Tipo
Orden de la matriz	N	entero
Matriz 4×4	mat	entero
Contador	i	entero
Contador	j	entero

Pseudocódigo

Inicio

Para i = N-1 hasta 0 hacer

 Para j = N-1 hasta 0 hacer

 Escribir mat[i][j]

 Fin Para

Fin Para

Fin

Código en C

```
#include <stdio.h>
int main() {
    int mat[4][4];
    int i, j;
    printf("Ingrese los elementos de la matriz:\n");
    for(i=0; i<4; i++) {
        for(j=0; j<4; j++) {
            scanf("%d",&mat[i][j]);
        }
    }
    printf("Matriz en sentido inverso:\n");

    for(i=3; i>=0; i--) {
        for(j=3; j>=0; j--) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Prueba de escritorio:

Entrada:

1 2

3 4

Proceso:

i=1, j=1 → 4

i=1, j=0 → 3

i=0, j=1 → 2

i=0, j=0 → 1

Salida:

4 3

2 1

Reflexión:

En este ejercicio se recorre la matriz desde la última fila y la última columna para mostrar los datos al revés. Hay que cuidar bien los índices para no salirse de los límites de la matriz.

Ejercicio 2.2.3.

Máximo de una fila

Enunciado. Escriba un programa que lea una matriz de N filas y N columnas de valores enteros. Luego debe pedir el número de una fila y mostrar por pantalla el valor de la mayor componente de esa fila.

Idea clave. Inicializar max con el primer elemento de la fila seleccionada y luego comparar con el resto de componentes de esa fila.

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz 4 x 4 componentes	mat	var	entero
Contador	i	var	entero
Contador	j	var	entero
Dato de entrada de fila	fil	var	entero
Máximo de la fila	max	var	entero
Uno y dos	1 y 2	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué es mejor iniciar max con un valor de la propia matriz y no con un número fijo como -1000?	

Tabla de objetos:

Objeto	Nombre	Tipo
Orden de la matriz	N	entero
Matriz 4×4 componentes	mat	entero
Contador	i	entero
Contador	j	entero
Dato de entrada de fila	fil	entero
Máximo de la fila	max	entero

Pseudocódigo

Inicio

Leer fil

max = mat[fil][0]

Para j = 1 hasta N-1 hacer

 Si mat[fil][j] > max entonces

 max = mat[fil][j]

 Fin Si

Fin Para

Escribir max

Fin

Código en C

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];
```

```
    int i, j, fil, max;
```

```
    printf("Ingrese los elementos de la matriz:\n");
```

```
    for(i=0; i<4; i++) {
```

```
        for(j=0; j<4; j++) {
```

```
            scanf("%d",&mat[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Ingrese la fila a revisar (0 a 3): ");
```

```
    scanf("%d",&fil);
```

```
    max = mat[fil][0];
```

```
    for(j=1; j<4; j++) {
```

```
        if(mat[fil][j] > max) {
```

```
            max = mat[fil][j];
```

```
        }
```

```
    }
```

```
    printf("El mayor valor de la fila es: %d", max);
```

```
    return 0;
```

```
}
```


Prueba de escritorio:

Entrada:

1 5 3 2

4 8 6 7

9 2 1 0

3 4 5 6

fil = 1

Proceso:

max = 4

j=1 → 8 > 4 → max=8

j=2 → 6 > 8 → no cambia

j=3 → 7 > 8 → no cambia

Salida:

8

Reflexión:

En este ejercicio se toma el primer valor de la fila como referencia y luego se compara con los demás elementos hasta encontrar el mayor.

Ejercicio 2.2.4.

Intercambiar las filas i, j de una matriz

Enunciado. Escriba un programa que intercambie las filas i y j de una matriz de enteros de N x N componentes, siendo i y j dos valores introducidos por teclado.

Idea clave. Usar una variable auxiliar para no perder información al intercambiar componente por componente.

Objeto	Nombre	Valor	Tipo
Orden de la matriz	N	cte	entero
Matriz de 4 x 4 componentes	mat	var	entero
Dato entrada primera fila	i	var	entero
Dato entrada segunda fila	j	var	entero
Contador	k	var	entero
Variable auxiliar	aux	var	entero
Uno	1	cte	entero
Aspecto de resolución		Orientación para el estudiante	
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Qué valor se perdería si se asigna directamente $mat[i][k] = mat[j][k]$ sin usar aux?	

Tabla de objetos:

Objeto	Nombre	Tipo
Orden de la matriz	N	entero
Matriz 4×4 componentes	mat	entero
Primera fila	i	entero
Segunda fila	j	entero
Contador	k	entero
Variable auxiliar	aux	entero

Pseudocódigo

Inicio

Leer i

Leer j

Para k = 0 hasta N-1 hacer

 aux = mat[i][k]

 mat[i][k] = mat[j][k]

 mat[j][k] = aux

Fin Para

Escribir matriz

Fin

Código en C

```
#include <stdio.h>
```

```
int main() {
```

```
    int mat[4][4];
```

```
    int i, j, k, aux;
```

```
    printf("Ingrese los elementos de la matriz:\n");
```

```
    for(i=0; i<4; i++) {
```

```
        for(j=0; j<4; j++) {
```

```
            scanf("%d",&mat[i][j]);
```

```
        }
```

```
    }
```

```
    printf("Ingrese la primera fila: ");
```

```
    scanf("%d",&i);
```

```
    printf("Ingrese la segunda fila: ");
```

```
    scanf("%d",&j);
```

```
    for(k=0; k<4; k++) {
```

```
        aux = mat[i][k];
```

```
        mat[i][k] = mat[j][k];
```

```
        mat[j][k] = aux;
```

```
    }
```

```
    printf("Matriz resultante:\n");
```

```
    for(i=0; i<4; i++) {
```

```
        for(j=0; j<4; j++) {
```

```
        printf("%d ", mat[i][j]);  
    }  
    printf("\n");  
}  
return 0;  
}
```

Prueba de escritorio:

Entrada:

1 2

3 4

i = 0

j = 1

Proceso:

aux = 1

mat[0][0] = 3

mat[1][0] = 1

aux = 2

mat[0][1] = 4

mat[1][1] = 2

Salida:

3 4

1 2

Reflexión:

En este ejercicio se usa una variable auxiliar para no perder datos al intercambiar las filas de la matriz. Sin ella, un valor podría sobrescribirse antes de completar el cambio.

Ejercicio 2.2.5.

Matriz traspuesta

Enunciado. Dada una matriz A de dimensión N x M, cuyas dimensiones reales y componentes se leen por teclado y por filas, calcule e imprima la matriz B de dimensión M x N, tal que $B = A^T$. Suponga que filas y columnas de A son menores o iguales a 10.

Idea clave. Al leer cada componente $A[i][j]$, almacenarla en $B[j][i]$. Luego imprimir B con dimensiones invertidas.

Objeto	Nombre	Valor	Tipo
Matriz de 10x10 componentes	A	var	real
Matriz de 10x10 componentes	B	var	real
Contador	i	var	entero
Contador	j	var	entero
Número de filas de la matriz A	N	var	entero
Número de columnas de la matriz A	M	var	entero
Uno	1	cte	entero
Objeto	Nombre	Valor	Tipo
Entrada		Identifica qué datos se leen por teclado y cuáles ya están definidos.	
Proceso		Describe el recorrido necesario con ciclos y las condiciones o acumulaciones requeridas.	
Salida		Especifica qué se debe mostrar y en qué orden.	
Pensamiento crítico		¿Por qué la matriz traspuesta cambia filas por columnas? ¿Cómo se modifican los límites al imprimir B?	

Tabla de objetos:

Objeto	Nombre	Tipo
Matriz de 10×10 componentes	A	real
Matriz de 10×10 componentes	B	real
Contador	i	entero
Contador	j	entero
Número de filas	N	entero
Número de columnas	M	entero

Pseudocódigo

Inicio

Leer N

Leer M

Para i = 0 hasta N-1 hacer

 Para j = 0 hasta M-1 hacer

 Leer A[i][j]

 B[j][i] = A[i][j]

 Fin Para

Fin Para

Escribir B

Fin

Código en C

```
#include <stdio.h>
int main() {
    float A[10][10], B[10][10];
    int N, M, i, j;
    printf("Ingrese el numero de filas: ");
    scanf("%d",&N);
    printf("Ingrese el numero de columnas: ");
    scanf("%d",&M);
    for(i=0; i<N; i++) {
        for(j=0; j<M; j++) {
            printf("Ingrese un valor: ");
            scanf("%f",&A[i][j]);
            B[j][i] = A[i][j];
        }
    }
    printf("Matriz traspuesta:\n");
    for(i=0; i<M; i++) {
        for(j=0; j<N; j++) {
            printf("%.0f ", B[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Prueba de escritorio:

Entrada:

1 2 3

4 5 6

Proceso:

$A[0][0]=1 \rightarrow B[0][0]=1$

$A[0][1]=2 \rightarrow B[1][0]=2$

$A[0][2]=3 \rightarrow B[2][0]=3$

$A[1][0]=4 \rightarrow B[0][1]=4$

$A[1][1]=5 \rightarrow B[1][1]=5$

$A[1][2]=6 \rightarrow B[2][1]=6$

Salida:

1 4

2 5

3 6

Reflexión:

En este ejercicio las filas pasan a ser columnas y las columnas pasan a ser filas. Por eso se intercambian los índices al guardar los datos en la nueva matriz.